Linear model and model fit

José G. Dias

Table of contents


```
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0
──
✓ dplyr     1.1.4     ✓ readr     2.1.5
✓ forcats   1.0.0     ✓ stringr   1.5.1
✓ ggplot2   3.5.1     ✓ tibble    3.2.1
✓ lubridate 1.9.3     ✓ tidyr     1.3.1
✓ purrr     1.0.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts()
──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
head(mtcars)
```

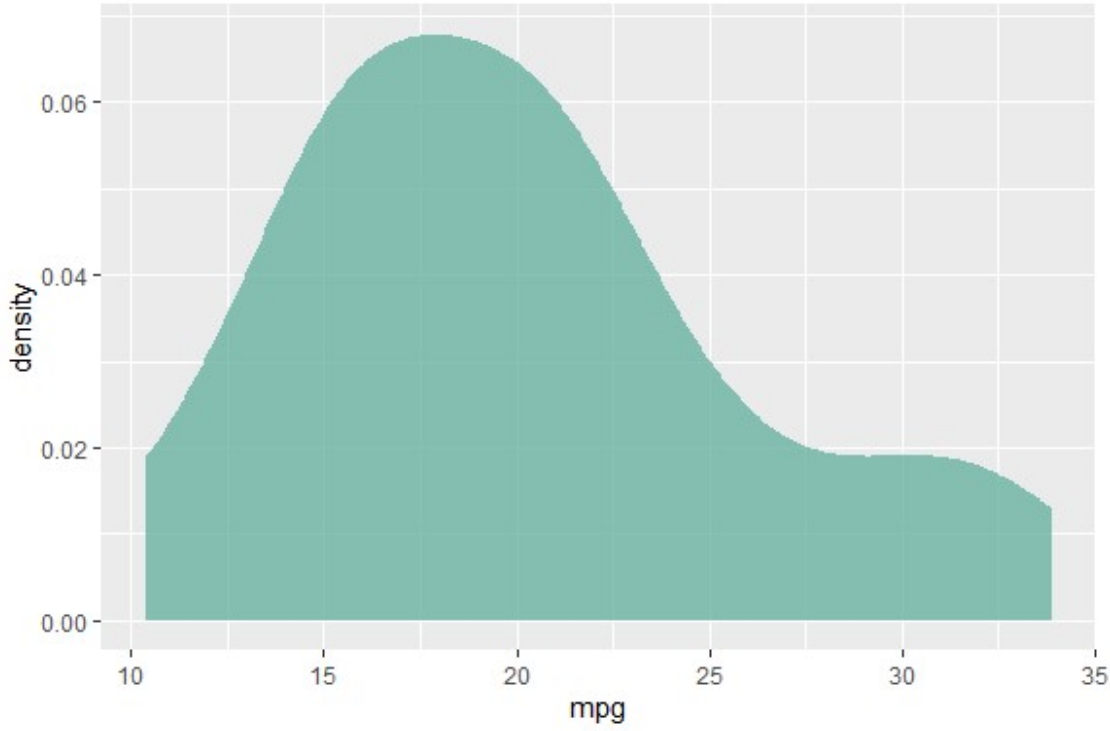
```
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```
set.seed(1234)
```

The goal of the lecture is to create a regression model that explains mpg as function of cyl and wt, i.e., $mpg = f(cyl, wt) + error$.

The data for dependent variable has this density:

```
ggplot(mtcars, aes(x=mpg)) +
  geom_density(fill="#69b3a2", color="#e9ecef", alpha=0.8)
```



3. Multiple linear regression (Normal errors)

3.1 Model specification

The for model to be defined is the (classic) multiple linear regression model with normal errors. Then, its specification is:

$$\mu(mpg_i) = \beta_0 + \beta_1 cyl_i + \beta_2 wt_i$$

$$mpg_i \sim N(\mu(mpg_i), \sigma^2)$$

Model estimation (OLS)

```
# Create a linear model of mpg values
#   DV = mpg, IVs = cyl, wt

lm_results <- lm(formula = mpg ~ cyl + wt, data = mtcars)

# Print summary statistics from mpg model
summary(lm_results)
```

```
Call:
lm(formula = mpg ~ cyl + wt, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-4.2893 -1.5512 -0.4684  1.5743  6.1004

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  39.6863     1.7150  23.141  < 2e-16 ***
cyl          -1.5078     0.4147  -3.636 0.001064 **
wt           -3.1910     0.7569  -4.216 0.000222 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.568 on 29 degrees of freedom
Multiple R-squared:  0.8302,    Adjusted R-squared:  0.8185
F-statistic: 70.91 on 2 and 29 DF,  p-value: 6.809e-12
```

3.2 Model estimation (MCMC)

```
#install.packages('rstan')
library(rstan)

Loading required package: StanHeaders


rstan version 2.32.6 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling
options(mc.cores = parallel::detectCores()).
To avoid recompilation of unchanged Stan programs, we recommend calling
rstan_options(auto_write = TRUE)
For within-chain threading using `reduce_sum()` or `map_rect()` Stan
functions,
change `threads_per_chain` option:
rstan_options(threads_per_chain = 1)

Do not specify '-march=native' in 'LOCAL_CPPFLAGS' or a Makevars file


Attaching package: 'rstan'

The following object is masked from 'package:tidyr':

    extract

library(posterior)

This is posterior version 1.5.0
```

```
Attaching package: 'posterior'

The following objects are masked from 'package:rstan':

    ess_bulk, ess_tail

The following objects are masked from 'package:stats':

    mad, sd, var

The following objects are masked from 'package:base':

    %in%, match
```

```
library(bayesplot)
```

```
This is bayesplot version 1.11.1

- Online documentation and vignettes at mc-stan.org/bayesplot

- bayesplot theme set to bayesplot::theme_default()

   * Does _not_ affect other ggplot2 plots

   * See ?bayesplot_theme_set for details on theme setting


Attaching package: 'bayesplot'

The following object is masked from 'package:posterior':

    rhat
```

```
library(coda)
```

```
Attaching package: 'coda'

The following object is masked from 'package:rstan':

    traceplot
```

```
color_scheme_set("brightblue")
```

```
# Data

# create X (add intercept column) and y
X           <- cbind(1, mtcars$cyl, mtcars$wt);
colnames(X) <- c('Intercept', 'cyl', 'wt')
y           <- mtcars$mpg
dat         <- list(N = length(mtcars$cyl), k = 3, y = y,X = X)
```

Define the normal regression model in Stan:

```
# Stan model code

stan_model.n <- '
data {                      // Data block; declarations only
  int<lower = 1> N;         // Sample size
  int<lower = 0> k;         // Dimension of model matrix
  matrix [N, k] X;          // Model Matrix
  vector[N] y;              // Target
}

parameters {                // Parameters block; declarations only
  vector[k] beta;           // Coefficient vector
  real<lower=0> tau;        // Precision parameter
}

//transformed parameters {    // Transformed parameters block; declarations
and statements.

//}

model {                     // Model block; declarations and statements.
  vector[N] mu;
  mu = X * beta;            // Linear predictor

  // Priors for betas
  beta  ~ normal(0, 100);

  // Prior for precision
  tau ~ gamma(1, 1);

  // likelihood
  y ~ normal(mu, sqrt(1/tau));
}

generated quantities {
  real<lower=0> sigma;  // Standard deviation

  // Calculate standard deviation from precision
  sigma = sqrt(1 / tau);
}'

# Running stan code
model.n <- stan_model(model_code = stan_model.n)

# Run the Stan model
```

```
stan_fit.n <- sampling(model.n, data = dat, chains = 4,
                       iter = 4000, warmup = 1000)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.000149 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take
1.49 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 1: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 1: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 1: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 1: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 1: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 1: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 1: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 1: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 1: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 1: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 1: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.385 seconds (Warm-up)
Chain 1:                0.602 seconds (Sampling)
Chain 1:                0.987 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 9e-06 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take
0.09 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 2: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 2: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 2: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 2: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 2: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 2: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 2: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 2: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 2: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 2: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%]  (Sampling)
```

```
Chain 2:
Chain 2:  Elapsed Time: 0.144 seconds (Warm-up)
Chain 2:                0.435 seconds (Sampling)
Chain 2:                0.579 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 1.4e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take
0.14 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 3: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 3: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 3: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 3: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 3: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 3: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 3: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 3: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 3: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 3: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.126 seconds (Warm-up)
Chain 3:                0.503 seconds (Sampling)
Chain 3:                0.629 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 1.1e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take
0.11 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 4: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 4: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 4: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 4: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 4: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 4: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 4: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 4: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 4: Iteration: 3400 / 4000 [ 85%]  (Sampling)
```

```
Chain 4: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 4: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.137 seconds (Warm-up)
Chain 4:                0.489 seconds (Sampling)
Chain 4:                0.626 seconds (Total)
Chain 4:
```

```
# Print and plot a summary of the Stan fit

print(stan_fit.n)
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=1000; thin=1;
post-warmup draws per chain=3000, total post-warmup draws=12000.

          mean se_mean   sd   2.5%    25%    50%    75%  97.5% n_eff Rhat
beta[1]  39.68    0.02 1.72  36.29  38.54  39.69  40.82  43.04  6869    1
beta[2]  -1.50    0.01 0.41  -2.30  -1.78  -1.51  -1.23  -0.68  5384    1
beta[3]  -3.20    0.01 0.75  -4.71  -3.69  -3.19  -2.69  -1.73  5673    1
tau       0.16    0.00 0.04   0.09   0.13   0.16   0.19   0.25  6586    1
sigma     2.55    0.00 0.34   1.99   2.31   2.52   2.75   3.33  6172    1
lp__    -48.73    0.02 1.49 -52.46 -49.45 -48.37 -47.64 -46.86  4124    1

Samples were drawn using NUTS(diag_e) at Wed Feb 26 16:17:47 2025.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

```
post.n <- as.array(stan_fit.n)
```

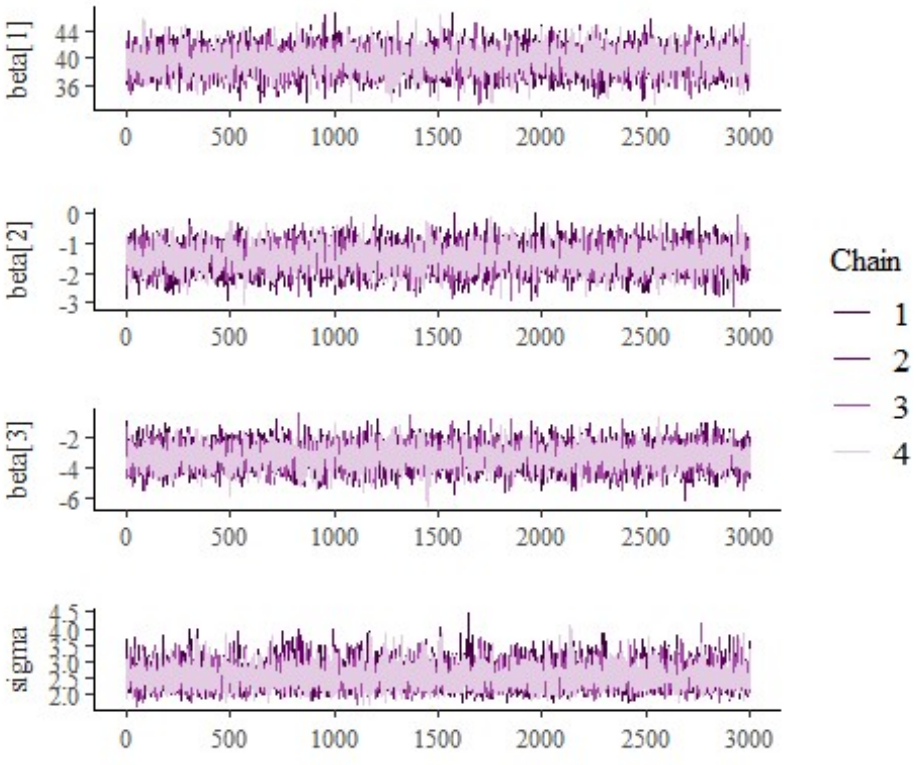
And the OLS estimates:

```
# Analyze results - classic statistics
summary(lm_results)$coefficients
```

```
             Estimate Std. Error   t value     Pr(>|t|)
(Intercept) 39.686261  1.7149840 23.140893 3.043182e-20
cyl         -1.507795  0.4146883 -3.635972 1.064282e-03
wt          -3.190972  0.7569065 -4.215808 2.220200e-04
```

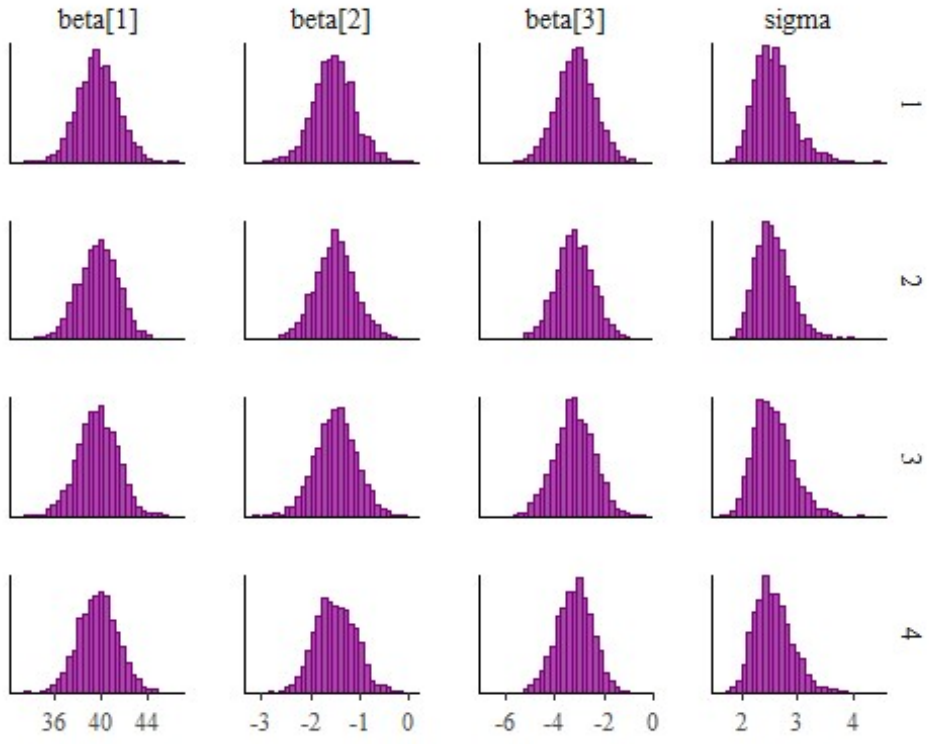
Now a quick look at the convergence:

```
#Quality
color_scheme_set("purple")
mcmc_trace(post.n, pars = c("beta[1]", "beta[2]", "beta[3]","sigma"),
           facet_args = list(ncol = 1, strip.position = "left"))
```

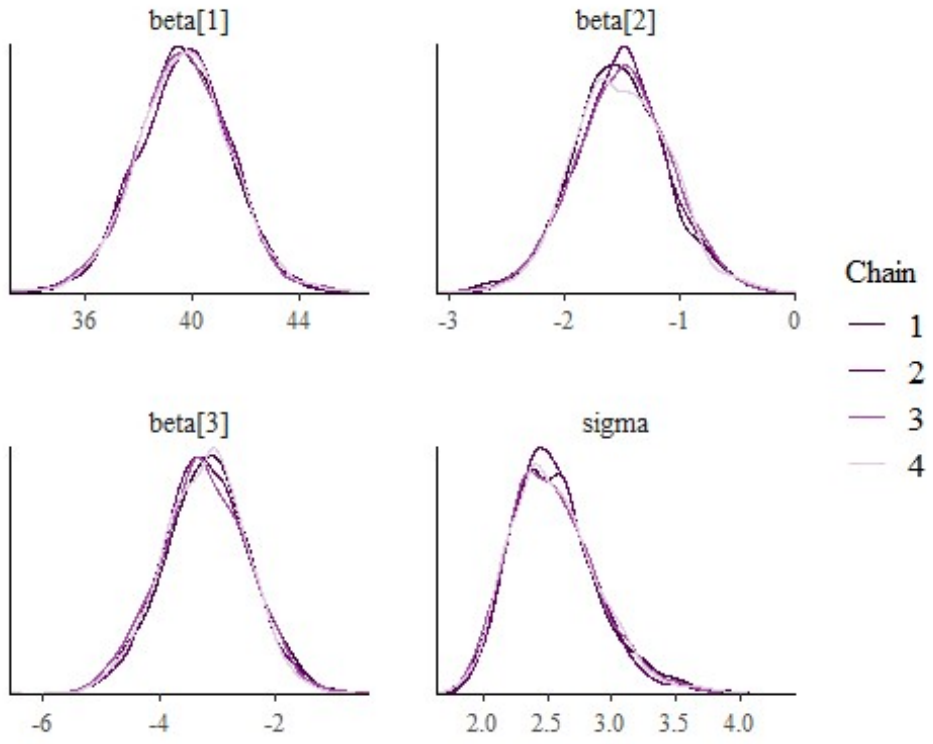



```
mcmc_hist_by_chain(post.n, pars = c("beta[1]", "beta[2]", "beta[3]","sigma"))
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
mcmc_dens_overlay(post.n, pars = c("beta[1]", "beta[2]", "beta[3]","sigma"))
```

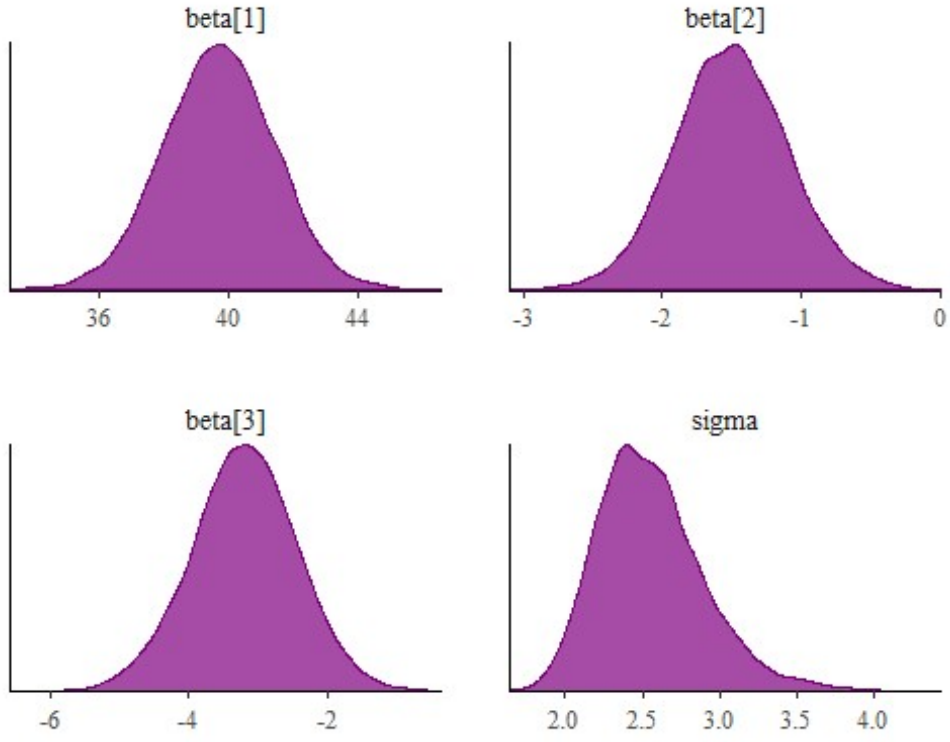



```
# Analyze results - Bayesian statistics

# Extract parameter names
param_names <- colnames(post.n)

# Compute mean and sd for each parameter
means <- colMeans(post.n)

# Display the results
param_statistics <- data.frame(Parameter = param_names, Mean = means[,1:5])
print(param_statistics)
```

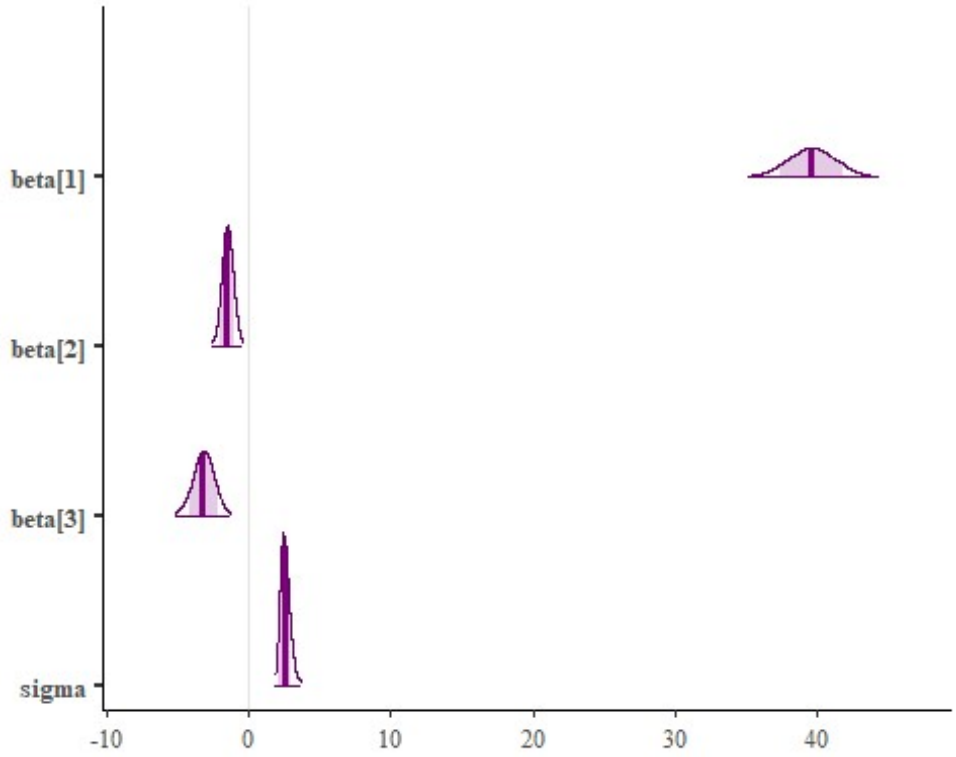
```
        Parameter Mean.beta.1. Mean.beta.2. Mean.beta.3.  Mean.tau Mean.sigma
chain:1   chain:1     39.68201    -1.526565    -3.155089 0.1606962   2.561045
chain:2   chain:2     39.70139    -1.499819    -3.206201 0.1605801   2.553233
chain:3   chain:3     39.65379    -1.488503    -3.217054 0.1624998   2.546701
chain:4   chain:4     39.70042    -1.498563    -3.213791 0.1610883   2.557315
```

And now the aggregate results:

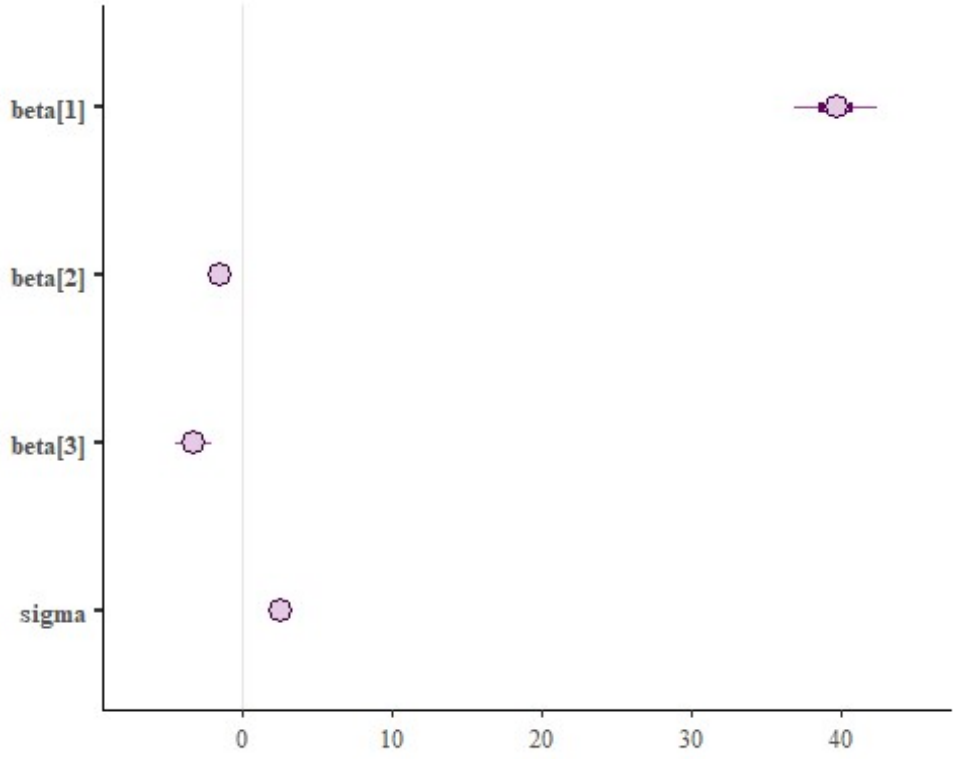
```
# Density plot
mcmc_dens(post.n, pars = c("beta[1]", "beta[2]", "beta[3]", "sigma"))
```



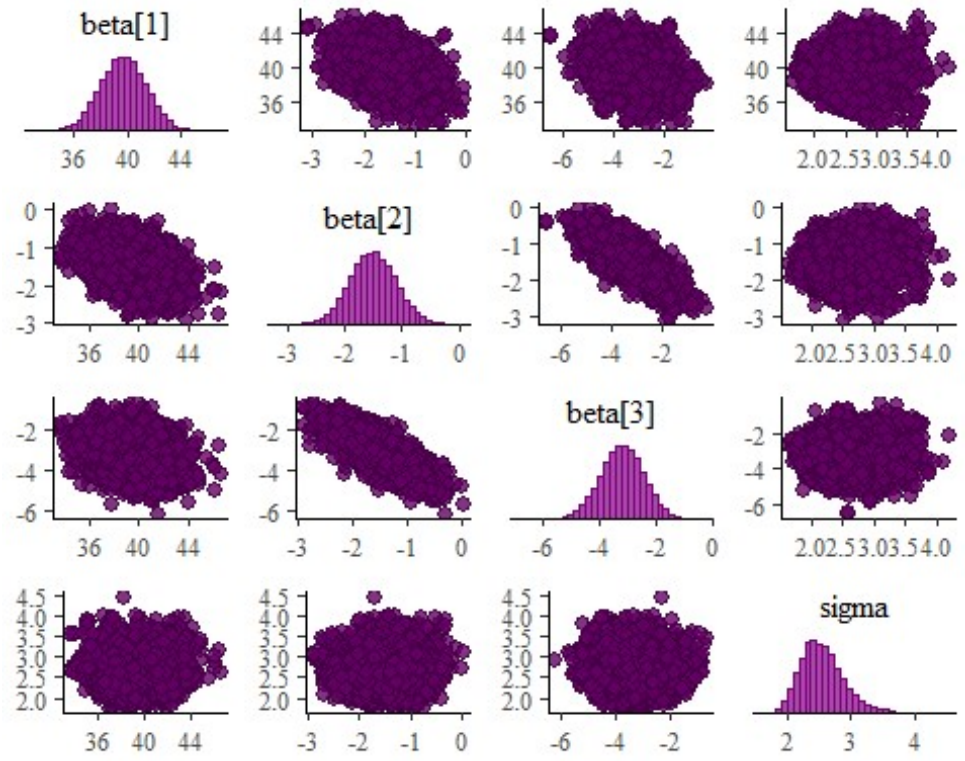

```
mcmc_areas(
  post.n,
  pars = c("beta[1]", "beta[2]", "beta[3]","sigma"),
  prob = 0.8, # 80% intervals
  prob_outer = 0.99, # 99%
  point_est = "mean"
)
```




```
mcmc_intervals(post.n, pars=c("beta[1]", "beta[2]", "beta[3]", "sigma"))
```



```
# Pairs plot
mcmc_pairs(post.n, pars = c("beta[1]", "beta[2]", "beta[3]","sigma"))
```



4. Model fit

Now let us explore a bit more model fit. How good is the model? We can use the block "generated quantities" in Stan to obtain extra insights for the last model.

4.1 Model estimation

```
# Stan model code

stan_model.modelfit <- '
data {                          // Data block; declarations only
  int<lower = 1> N;             // Sample size
  int<lower = 0> k;             // Dimension of model matrix
  matrix [N, k] X;              // Model Matrix
  vector[N] y;                  // Target
}

parameters {                    // Parameters block; declarations only
  vector[k] beta;               // Coefficient vector
  real<lower=0> tau;            // Precision parameter
}

//transformed parameters {     // Transformed parameters block; declarations
```

```
and statements.

//}

model {                     // Model block; declarations and statements.
  vector[N] mu;
  mu = X * beta;            // Linear predictor

  // Priors for betas
  beta  ~ normal(0, 100);

  // Prior for precision
  tau ~ gamma(1, 1);

  // likelihood
  y ~ normal(mu, sqrt(1/tau));
}

generated quantities {
  real<lower=0> sigma;  // Standard deviation
  real ess;
  real totalss;
  real R2;                  // Calculate Rsq
  vector[N] y_hat;

  // Calculate standard deviation from precision
  sigma = sqrt(1 / tau);

  // Calculate Rsq
  y_hat = X * beta;
  ess = dot_self(y - y_hat);
  totalss = dot_self(y - mean(y));
  R2 = 1 - ess/totalss;
}'

# Running stan code
model.modelfit <- stan_model(model_code = stan_model.modelfit)

# Run the Stan model

stan_fit.modelfit<- sampling(model.modelfit, data = dat, chains = 4,
                      iter = 4000, warmup = 1000, thin  = 4, verbose =
FALSE)


SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.00017 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take
```

```
1.7 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 1: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 1: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 1: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 1: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 1: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 1: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 1: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 1: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 1: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 1: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 1: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.27 seconds (Warm-up)
Chain 1:                0.582 seconds (Sampling)
Chain 1:                0.852 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.1e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take
0.11 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 2: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 2: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 2: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 2: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 2: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 2: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 2: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 2: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 2: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 2: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.188 seconds (Warm-up)
Chain 2:                0.603 seconds (Sampling)
Chain 2:                0.791 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
```

```
Chain 3: Gradient evaluation took 1.8e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take
0.18 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 3: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 3: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 3: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 3: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 3: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 3: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 3: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 3: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 3: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 3: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.224 seconds (Warm-up)
Chain 3:                0.393 seconds (Sampling)
Chain 3:                0.617 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 1.3e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take
0.13 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 4: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 4: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 4: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 4: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 4: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 4: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 4: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 4: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 4: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 4: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 4: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.147 seconds (Warm-up)
Chain 4:                0.442 seconds (Sampling)
Chain 4:                0.589 seconds (Total)
Chain 4:
```

```
post.modelfit <- as.array(stan_fit.modelfit)

# Print and plot a summary of the Stan fit
print(stan_fit.modelfit)
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=1000; thin=4;
post-warmup draws per chain=750, total post-warmup draws=3000.

             mean se_mean    sd    2.5%     25%     50%     75%   97.5% n_eff
beta[1]     39.63    0.03  1.72   36.15   38.51   39.65   40.79   43.04  2711
beta[2]     -1.50    0.01  0.41   -2.31   -1.77   -1.51   -1.24   -0.72  2899
beta[3]     -3.19    0.01  0.74   -4.65   -3.68   -3.18   -2.70   -1.73  2791
tau          0.16    0.00  0.04    0.09    0.13    0.16    0.18    0.25  2974
sigma        2.56    0.01  0.34    1.99    2.33    2.52    2.76    3.32  2949
ess        210.73    0.36 17.61  192.48  198.34  205.70  217.22  257.67  2418
totalss   1126.05    0.00  0.00 1126.05 1126.05 1126.05 1126.05 1126.05     2
R2           0.81    0.00  0.02    0.77    0.81    0.82    0.82    0.83  2418
y_hat[1]    22.26    0.01  0.60   21.05   21.88   22.26   22.66   23.43  2936
y_hat[2]    21.45    0.01  0.50   20.46   21.13   21.45   21.79   22.41  2990
y_hat[3]    26.23    0.01  0.72   24.77   25.74   26.24   26.72   27.64  2832
y_hat[4]    20.37    0.01  0.46   19.45   20.07   20.37   20.67   21.29  3003
y_hat[5]    16.65    0.01  0.76   15.16   16.15   16.64   17.13   18.16  3001
y_hat[6]    19.59    0.01  0.52   18.58   19.24   19.58   19.93   20.62  2945
y_hat[7]    16.23    0.01  0.72   14.83   15.76   16.23   16.69   17.63  3014
y_hat[8]    23.45    0.02  0.98   21.51   22.79   23.45   24.11   25.38  2848
y_hat[9]    23.58    0.02  0.96   21.66   22.93   23.57   24.23   25.46  2849
y_hat[10]   19.65    0.01  0.51   18.66   19.31   19.65   20.00   20.67  2950
y_hat[11]   19.65    0.01  0.51   18.66   19.31   19.65   20.00   20.67  2950
y_hat[12]   14.64    0.01  0.66   13.34   14.22   14.63   15.07   15.92  3010
y_hat[13]   15.72    0.01  0.68   14.40   15.28   15.71   16.16   17.09  3025
y_hat[14]   15.56    0.01  0.67   14.26   15.13   15.56   16.00   16.91  3027
y_hat[15]   10.88    0.02  1.15    8.67   10.14   10.86   11.63   13.17  2865
y_hat[16]   10.33    0.02  1.26    7.91    9.51   10.32   11.15   12.81  2858
y_hat[17]   10.58    0.02  1.21    8.26    9.79   10.56   11.37   12.96  2861
y_hat[18]   26.61    0.01  0.73   25.11   26.12   26.63   27.09   28.03  2813
y_hat[19]   28.47    0.02  0.88   26.66   27.90   28.49   29.05   30.17  2764
y_hat[20]   27.77    0.02  0.80   26.13   27.26   27.80   28.30   29.35  2757
y_hat[21]   25.76    0.01  0.74   24.31   25.28   25.77   26.27   27.21  2851
y_hat[22]   16.39    0.01  0.73   14.94   15.91   16.39   16.86   17.85  3010
y_hat[23]   16.66    0.01  0.77   15.17   16.17   16.65   17.14   18.18  3001
y_hat[24]   15.37    0.01  0.66   14.07   14.94   15.36   15.81   16.68  3027
y_hat[25]   15.36    0.01  0.66   14.05   14.93   15.35   15.79   16.66  3026
y_hat[26]   27.45    0.01  0.77   25.88   26.96   27.48   27.96   28.97  2771
y_hat[27]   26.80    0.01  0.73   25.29   26.32   26.82   27.28   28.25  2803
y_hat[28]   28.80    0.02  0.93   26.94   28.20   28.80   29.41   30.58  2771
y_hat[29]   17.51    0.02  0.89   15.79   16.94   17.50   18.07   19.25  2971
y_hat[30]   21.79    0.01  0.53   20.72   21.45   21.78   22.14   22.83  2967
y_hat[31]   16.23    0.01  0.72   14.83   15.76   16.23   16.69   17.63  3014
y_hat[32]   24.76    0.02  0.81   23.13   24.22   24.76   25.30   26.35  2862
```

```
lp__       -48.71    0.03  1.50  -52.47  -49.45  -48.35  -47.60  -46.86  2557
          Rhat
beta[1]      1
beta[2]      1
beta[3]      1
tau          1
sigma        1
ess          1
totalss      1
R2           1
y_hat[1]     1
y_hat[2]     1
y_hat[3]     1
y_hat[4]     1
y_hat[5]     1
y_hat[6]     1
y_hat[7]     1
y_hat[8]     1
y_hat[9]     1
y_hat[10]    1
y_hat[11]    1
y_hat[12]    1
y_hat[13]    1
y_hat[14]    1
y_hat[15]    1
y_hat[16]    1
y_hat[17]    1
y_hat[18]    1
y_hat[19]    1
y_hat[20]    1
y_hat[21]    1
y_hat[22]    1
y_hat[23]    1
y_hat[24]    1
y_hat[25]    1
y_hat[26]    1
y_hat[27]    1
y_hat[28]    1
y_hat[29]    1
y_hat[30]    1
y_hat[31]    1
y_hat[32]    1
lp__         1

Samples were drawn using NUTS(diag_e) at Wed Feb 26 16:19:28 2025.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

4.2 Model fit

Now we can make a selection of the important parameters we want to analyze, otherwise all parameters and quantities will be printed (e.g., y_hat, i.e. expected values):

```
print(stan_fit.modelfit, digits_summary = 3,
      pars  = c("beta[1]", "beta[2]", "beta[3]", "sigma", "R2"),
      probs = c(.025, .5, .975)
)
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=1000; thin=4;
post-warmup draws per chain=750, total post-warmup draws=3000.

          mean se_mean    sd   2.5%    50%  97.5% n_eff  Rhat
beta[1] 39.628   0.033 1.720 36.145 39.648 43.038  2711 1.000
beta[2] -1.503   0.008 0.405 -2.306 -1.509 -0.716  2899 1.000
beta[3] -3.185   0.014 0.744 -4.651 -3.182 -1.725  2791 1.001
sigma    2.558   0.006 0.336  1.995  2.522  3.318  2949 1.001
R2       0.813   0.000 0.016  0.771  0.817  0.829  2418 0.999

Samples were drawn using NUTS(diag_e) at Wed Feb 26 16:19:28 2025.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

And from the lm, we have:

```
# Print summary statistics from mpg model
summary(lm_results)
```

```
Call:
lm(formula = mpg ~ cyl + wt, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-4.2893 -1.5512 -0.4684  1.5743  6.1004

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  39.6863     1.7150  23.141  < 2e-16 ***
cyl          -1.5078     0.4147  -3.636 0.001064 **
wt           -3.1910     0.7569  -4.216 0.000222 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.568 on 29 degrees of freedom
Multiple R-squared:  0.8302,    Adjusted R-squared:  0.8185
F-statistic: 70.91 on 2 and 29 DF,  p-value: 6.809e-12
```

These results can be further explored, for instance compare y and $\hat{(y)}$. Nevertheless, we will explore instead the posterior predictive distribution.

5. Posterior predictive distribution

The posterior predictive distribution is given by:

$$p(\tilde{y}|y) = \int p(\tilde{y}|\theta)p(\theta|y)d\theta,$$

where the posterior distribution weights the results.

```
# Stan model code

stan_model.pp <- '
data {                      // Data block; declarations only
  int<lower = 1> N;         // Sample size
  int<lower = 0> k;         // Dimension of model matrix
  matrix [N, k] X;          // Model Matrix
  vector[N] y;              // Target
}

parameters {                // Parameters block; declarations only
  vector[k] beta;           // Coefficient vector
  real<lower=0> tau;        // Precision parameter
}

transformed parameters {    // Transformed parameters block; declarations and
statements.
  vector[N] mu;
  mu = X * beta;            // Linear predictor
}

model {                     // Model block; declarations and statements.
  // Priors for betas
  beta  ~ normal(0, 100);

  // Prior for precision
  tau ~ gamma(1, 1);

  // likelihood
  y ~ normal(mu, sqrt(1/tau));
}

generated quantities {

  // Calculate standard deviation from precision
  real<lower=0> sigma;  // Standard deviation
  sigma = sqrt(1 / tau);
```

```
  // Generate posterior predictive samples
  real y_pred[N];    // Posterior predictive samples
  for (i in 1:N)
    y_pred[i] = normal_rng(mu[i], sigma);
}'

# Running stan code
model.pp <- stan_model(model_code = stan_model.pp)

# Run the Stan model

stan_fit.pp<- sampling(model.pp, data = dat, chains = 4,
                       iter = 4000, warmup = 1000, thin  = 4, verbose =
FALSE)


SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 5.4e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take
0.54 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 1: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 1: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 1: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 1: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 1: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 1: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 1: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 1: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 1: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 1: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 1: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.221 seconds (Warm-up)
Chain 1:                0.584 seconds (Sampling)
Chain 1:                0.805 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.4e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take
0.14 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
```

```
Chain 2:
Chain 2: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 2: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 2: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 2: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 2: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 2: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 2: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 2: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 2: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 2: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 2: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.188 seconds (Warm-up)
Chain 2:                0.435 seconds (Sampling)
Chain 2:                0.623 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 1.1e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take
0.11 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 3: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 3: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 3: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 3: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 3: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 3: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 3: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 3: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 3: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 3: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.179 seconds (Warm-up)
Chain 3:                0.471 seconds (Sampling)
Chain 3:                0.65 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 7e-06 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take
0.07 seconds.
```

```
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 4: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 4: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 4: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 4: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 4: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 4: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 4: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 4: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 4: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 4: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 4: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.206 seconds (Warm-up)
Chain 4:                0.563 seconds (Sampling)
Chain 4:                0.769 seconds (Total)
Chain 4:
```

```r
post.modelfit <- as.array(stan_fit.pp)

# Print and plot a summary of the Stan fit
print(stan_fit.pp)
```

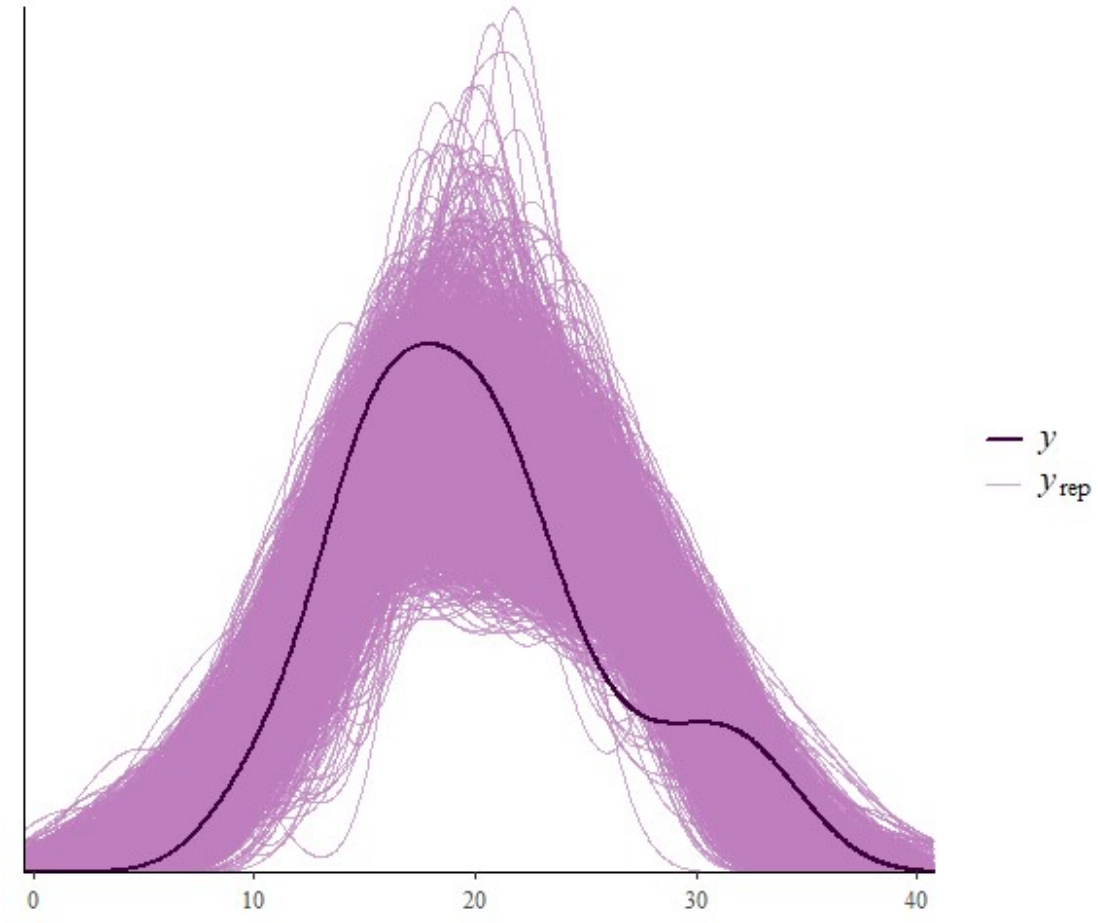
```
Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=1000; thin=4;
post-warmup draws per chain=750, total post-warmup draws=3000.

            mean se_mean   sd   2.5%    25%    50%    75%  97.5% n_eff Rhat
beta[1]    39.63    0.03 1.73  36.19  38.46  39.66  40.78  43.04  2937    1
beta[2]    -1.50    0.01 0.41  -2.31  -1.78  -1.50  -1.23  -0.68  2771    1
beta[3]    -3.19    0.01 0.76  -4.66  -3.69  -3.20  -2.68  -1.69  2616    1
tau         0.16    0.00 0.04   0.09   0.13   0.16   0.19   0.25  2550    1
mu[1]      22.27    0.01 0.60  21.11  21.88  22.26  22.66  23.44  3034    1
mu[2]      21.46    0.01 0.49  20.50  21.13  21.45  21.78  22.42  3020    1
mu[3]      26.23    0.01 0.73  24.78  25.77  26.24  26.70  27.61  2922    1
mu[4]      20.38    0.01 0.46  19.51  20.06  20.37  20.68  21.30  2944    1
mu[5]      16.66    0.01 0.77  15.14  16.15  16.65  17.16  18.22  2961    1
mu[6]      19.60    0.01 0.52  18.59  19.26  19.59  19.94  20.65  2893    1
mu[7]      16.24    0.01 0.72  14.83  15.76  16.24  16.71  17.71  2975    1
mu[8]      23.46    0.02 1.00  21.50  22.80  23.46  24.10  25.52  2695    1
mu[9]      23.59    0.02 0.98  21.67  22.94  23.59  24.21  25.60  2703    1
mu[10]     19.66    0.01 0.51  18.66  19.32  19.65  20.00  20.69  2896    1
mu[11]     19.66    0.01 0.51  18.66  19.32  19.65  20.00  20.69  2896    1
mu[12]     14.65    0.01 0.66  13.34  14.21  14.66  15.07  15.96  2960    1
mu[13]     15.73    0.01 0.68  14.42  15.29  15.73  16.17  17.06  2979    1
mu[14]     15.57    0.01 0.67  14.26  15.14  15.57  16.00  16.90  2977    1
mu[15]     10.89    0.02 1.16   8.61  10.11  10.89  11.65  13.29  2680    1
```

mu[16]	10.34	0.02	1.27	7.83	9.48	10.34	11.16	12.96	2634	1
mu[17]	10.59	0.02	1.22	8.18	9.77	10.59	11.37	13.11	2654	1
mu[18]	26.61	0.01	0.73	25.15	26.13	26.62	27.09	28.01	2940	1
mu[19]	28.48	0.02	0.88	26.72	27.89	28.51	29.06	30.17	2999	1
mu[20]	27.78	0.01	0.80	26.16	27.24	27.80	28.30	29.31	2985	1
mu[21]	25.77	0.01	0.74	24.31	25.30	25.78	26.26	27.16	2901	1
mu[22]	16.40	0.01	0.74	14.94	15.92	16.40	16.89	17.89	2970	1
mu[23]	16.67	0.01	0.77	15.15	16.16	16.67	17.18	18.23	2960	1
mu[24]	15.38	0.01	0.66	14.09	14.95	15.38	15.81	16.71	2975	1
mu[25]	15.37	0.01	0.66	14.08	14.94	15.37	15.79	16.69	2974	1
mu[26]	27.46	0.01	0.77	25.88	26.94	27.49	27.96	28.93	2975	1
mu[27]	26.80	0.01	0.73	25.32	26.32	26.81	27.29	28.20	2949	1
mu[28]	28.80	0.02	0.93	26.97	28.19	28.84	29.41	30.61	3003	1
mu[29]	17.52	0.02	0.90	15.73	16.94	17.52	18.11	19.34	2931	1
mu[30]	21.79	0.01	0.53	20.77	21.44	21.79	22.15	22.83	3030	1
mu[31]	16.24	0.01	0.72	14.83	15.76	16.24	16.71	17.71	2975	1
mu[32]	24.77	0.02	0.82	23.17	24.24	24.77	25.28	26.40	2822	1
sigma	2.55	0.01	0.33	2.00	2.31	2.52	2.75	3.30	2498	1
y_pred[1]	22.20	0.05	2.62	17.07	20.44	22.24	23.97	27.21	2977	1
y_pred[2]	21.43	0.05	2.57	16.34	19.70	21.43	23.13	26.36	2980	1
y_pred[3]	26.19	0.05	2.66	21.14	24.40	26.13	27.97	31.51	2722	1
y_pred[4]	20.43	0.05	2.57	15.38	18.72	20.44	22.09	25.34	3013	1
y_pred[5]	16.67	0.05	2.69	11.46	14.83	16.70	18.43	22.10	2735	1
y_pred[6]	19.48	0.05	2.56	14.46	17.78	19.48	21.18	24.48	2554	1
y_pred[7]	16.17	0.05	2.59	11.20	14.44	16.13	17.88	21.41	2776	1
y_pred[8]	23.49	0.05	2.76	17.93	21.67	23.49	25.31	28.83	2812	1
y_pred[9]	23.62	0.05	2.72	18.45	21.77	23.58	25.44	29.05	2898	1
y_pred[10]	19.72	0.05	2.60	14.49	18.00	19.69	21.49	24.80	2667	1
y_pred[11]	19.80	0.05	2.63	14.57	18.08	19.74	21.50	25.03	2966	1
y_pred[12]	14.67	0.05	2.72	9.27	12.89	14.65	16.46	20.10	2980	1
y_pred[13]	15.65	0.05	2.68	10.56	13.82	15.66	17.42	20.88	2926	1
y_pred[14]	15.62	0.05	2.64	10.38	13.90	15.59	17.31	20.74	3043	1
y_pred[15]	10.83	0.05	2.81	5.27	8.96	10.84	12.70	16.47	2645	1
y_pred[16]	10.25	0.05	2.88	4.80	8.35	10.13	12.19	16.02	3342	1
y_pred[17]	10.60	0.05	2.80	5.25	8.70	10.60	12.50	16.01	3079	1
y_pred[18]	26.66	0.05	2.67	21.34	25.04	26.70	28.37	31.98	2980	1
y_pred[19]	28.52	0.05	2.75	23.17	26.71	28.60	30.33	33.78	2829	1
y_pred[20]	27.78	0.05	2.66	22.53	26.01	27.76	29.63	32.85	3096	1
y_pred[21]	25.86	0.05	2.74	20.76	23.95	25.83	27.67	31.14	3184	1
y_pred[22]	16.38	0.05	2.73	10.97	14.63	16.42	18.17	21.61	2813	1
y_pred[23]	16.66	0.05	2.68	11.50	14.87	16.65	18.40	21.93	3042	1
y_pred[24]	15.41	0.05	2.64	10.29	13.66	15.42	17.12	20.62	2877	1
y_pred[25]	15.37	0.05	2.66	10.14	13.68	15.33	17.10	20.79	2963	1
y_pred[26]	27.46	0.05	2.68	22.37	25.67	27.45	29.21	32.68	3051	1
y_pred[27]	26.68	0.05	2.69	21.41	24.95	26.70	28.48	31.92	3140	1
y_pred[28]	28.82	0.05	2.73	23.34	27.08	28.84	30.57	34.22	3340	1
y_pred[29]	17.56	0.05	2.74	12.47	15.77	17.59	19.36	23.29	2783	1
y_pred[30]	21.70	0.05	2.64	16.70	19.94	21.69	23.47	27.03	2917	1
y_pred[31]	16.20	0.05	2.63	11.01	14.49	16.19	17.96	21.42	2983	1
y_pred[32]	24.71	0.05	2.75	19.46	22.88	24.65	26.48	30.24	2932	1

```
lp__       -48.71    0.03 1.48 -52.50 -49.47 -48.37 -47.63 -46.87  2700    1

Samples were drawn using NUTS(diag_e) at Wed Feb 26 16:20:15 2025.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

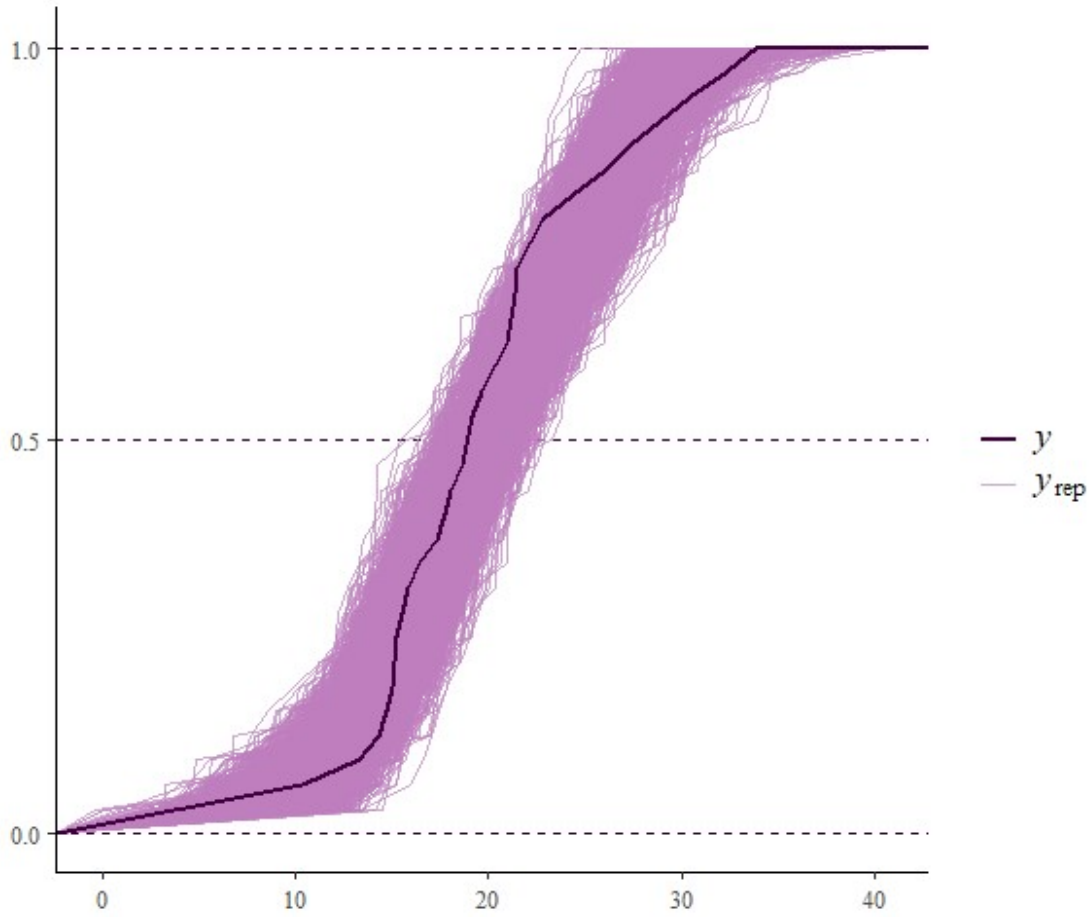
Visualize the posterior predictive distribution.

```
y_rep <- rstan::extract(stan_fit.pp, par = 'y_pred')$y_pred
ppc_dens_overlay(y,y_rep)
```



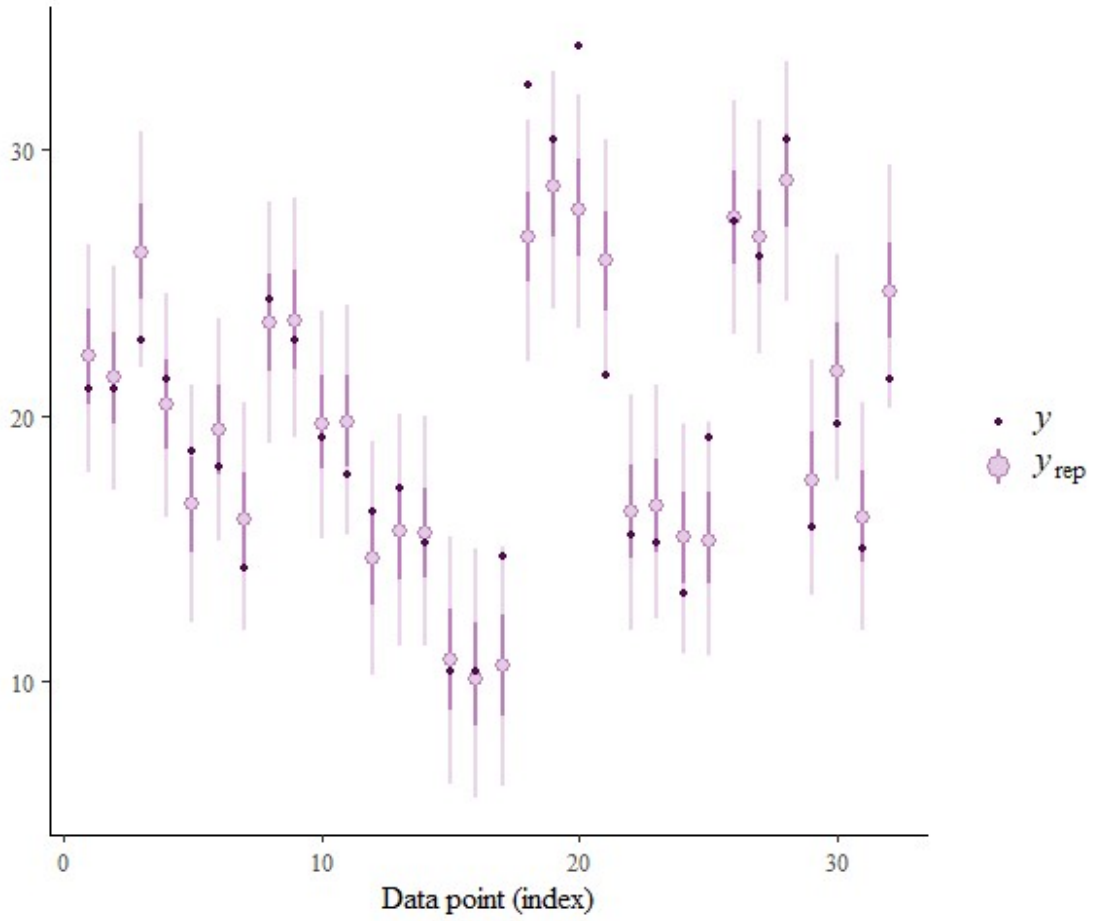
And we can depict the empirical distribution function:

```
ppc_ecdf_overlay(y,y_rep)
```



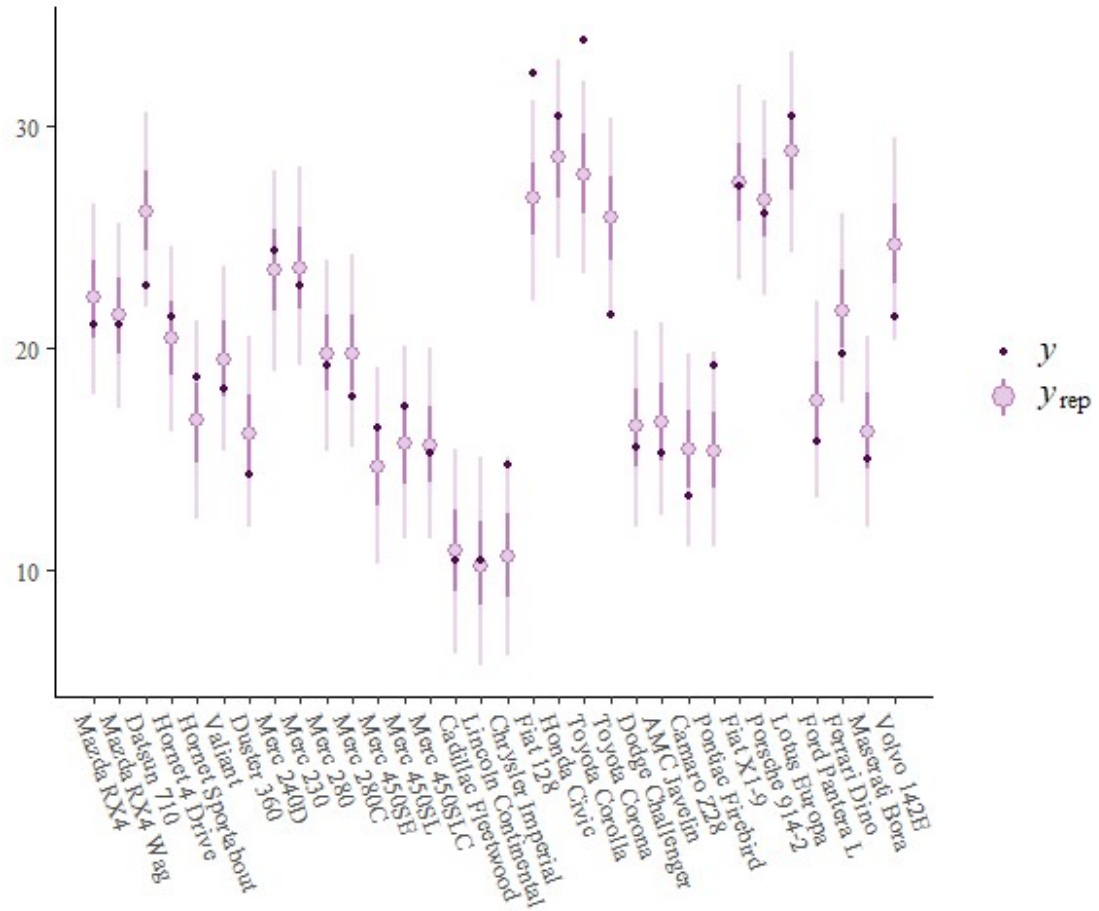
And for each observation:

```
ppc_intervals(y,y_rep)
```



We can go further in trying to understand the data set (e.g., include dummy variables in the model). In the next example we add the label of each observation to the previous graph.

```
ppc_intervals(y, y_rep,prob=0.5) +
  scale_x_continuous(labels = rownames(mtcars),breaks = 1:nrow(mtcars)) +
  xaxis_text(angle = -70, vjust = 1, hjust = 0) +
  xaxis_title(FALSE)
```



5.1 Bayesian p-value

A Bayesian p-value is a measure of the discrepancy between observed data and data simulated from the posterior predictive distribution. By comparing the observed test statistic with the distribution of test statistics generated from the posterior predictive distribution, you can assess whether the model represents the data (assuming the MCMC chains have converged to the posterior distribution). The Bayesian p-value is the proportion of posterior predictive samples that are more extreme (i.e., have a larger discrepancy) than the observed test statistic. P-values close to 0.5 suggest good fit.

After obtaining the posterior samples, we generate data simulated data from the posterior predictive distribution, and then a test statistic. Then we can compare with the test statistic for the observed data.

Let us illustrate first for the mean statistic:

```
ppc_stat(y,y_rep)
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

And median:

```
ppc_stat(y,y_rep, stat = "median")
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

Now we can compute the p-value (based on T()= “mean”:

```
# Extract posterior predictive samples
posterior_predictive_samples <- y_rep

# Compute test statistic for observed data
observed_test_statistic <- mean(y)

# Compute test statistic for each posterior predictive sample
posterior_predictive_test_statistics <- apply(y_rep, 1, mean)

# Calculate Bayesian p-value for each posterior predictive sample
bayesian_p_values <- sapply(posterior_predictive_test_statistics,
function(stat) {
    (stat >= observed_test_statistic)
})

# Plot Bayesian p-values and assess convergence
print(mean(bayesian_p_values))
```

```
[1] 0.4893333
```